

MỤC LỤC

1	Tổng quan về hệ thống	2
1.1	Sự cần thiết	2
1.2	Khảo sát công nghệ xây dựng cốt lõi	2
1.2.1	Mô hình ngôn ngữ lớn LLM (Large Language Model)	2
1.2.2	Tạo tăng cường truy xuất RAG (Retrieval Augmented Generation)	4
1.2.3	Kiến trúc vi dịch vụ MSA (Microservice Architecture)	7
1.2.4	Thiết kế hướng miền DDD (Domain Driven Design)	8
	TÀI LIỆU THAM KHẢO	11

1 Tổng quan về hệ thống

Trong thời đại công nghệ hiện nay, việc ứng dụng công nghệ vào cuộc sống trở thành một phần không thể thiếu. Đặc biệt trong thời đại công nghệ trí tuệ nhân tạo (AI) đang bùng nổ. Mục tiêu của ứng dụng này là phát triển một hệ thống hỏi đáp về pháp luật Việt Nam. Việc ứng dụng công nghệ AI đánh dấu sự chuyển dịch lớn của ngành pháp lý từ sử dụng các công cụ, phương thức tìm kiếm thủ công sang hệ thống xử lý ngôn ngữ tự nhiên (NLP). Việc ứng dụng công nghệ xử lý ngôn ngữ tự nhiên giúp máy tính hiểu và xử lý được ngôn ngữ của con người.

1.1 Sự cần thiết

Hệ thống tài liệu văn bản pháp luật rất phức tạp. Sự phức tạp không chỉ nằm ở khối lượng dữ liệu, mà các văn bản có thể thay thế, bổ sung, hết hiệu lực, ... Trong một số trường hợp một văn bản vừa hết hiệu lực hoặc bị thay thế thì con người có thể chưa kịp cập nhật thông tin. Đặc biệt là đối với những người bận rộn. Từ đó, có thể dẫn đến những sai sót, thậm chí là thiệt hại về tài chính và rủi ro pháp lý.



Hình 1.1: AI hỗ trợ tư vấn pháp luật

Đối với ứng dụng AI thì sẽ có các điểm mạnh như sau:

- Tiết kiệm chi phí
- Hoạt động làm việc 24/7
- Phản hồi nhanh chóng
- Cập nhật thông tin thường xuyên
- Trích dẫn thông tin chi tiết

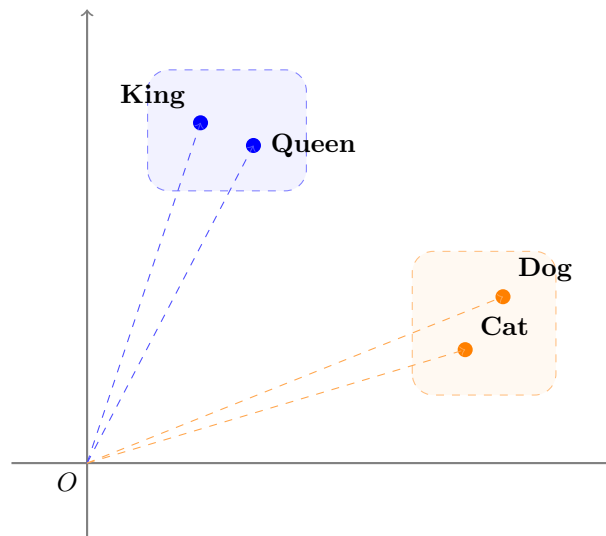
Vì AI phụ thuộc vào dữ liệu đầu vào, do việc cập nhật dữ liệu không được ngay lập tức, nên không đảm bảo tính chính xác của các văn bản pháp luật dẫn đến hạn chế và thách thức về độ chính xác của ứng dụng. Vì vậy, trong môi trường đòi hỏi sự chính xác tuyệt đối như pháp luật thì công cụ AI chỉ đóng vai trò trợ lý hỗ trợ, nguồn tham khảo và không thể thay thế con người. Do đó, để sử dụng hiệu quả và tránh những rủi ro sai lệch thông tin, người dùng phải kiểm chứng thông tin với chuyên gia và các CSDL chính thống trong các vấn đề mang tính quyết định quan trọng.

1.2 Khảo sát công nghệ xây dựng cốt lõi

1.2.1 Mô hình ngôn ngữ lớn LLM (Large Language Model)

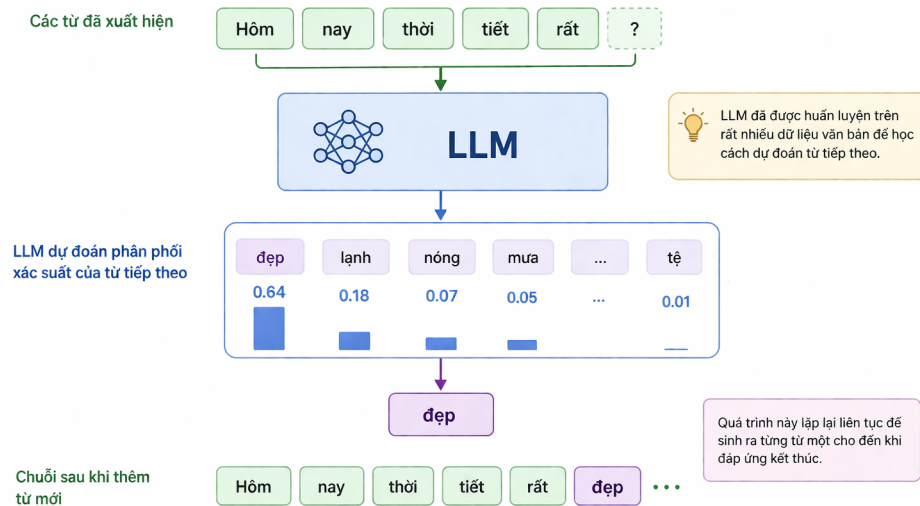
Năm 2017, các kỹ sư của Google đưa ra bài báo "Attention Is All You Need" khai sinh kiến trúc Transformer với cơ chế Self-Attention (Tự chú ý), mở đường cho hầu hết kiến trúc LLM (Large Language Model) hiện nay. Cách hoạt động của LLM là dự đoán xác suất của từ tiếp theo dựa theo các từ đã xuất hiện trước đó. LLM không đọc chữ như con người. Chia câu thành các phần nhỏ gọi là token. Biến mỗi token thành vector trong không gian nhiều chiều. Những từ có nghĩa

gần nhau vua (king) và hoàng hậu (queen), hoặc mèo (cat) và chó (dog) sẽ có các tọa độ rất gần nhau trong không gian.



Hình 1.2: Mô phỏng quá trình ánh xạ các vector từ gần nghĩa.

Khi dự đoán từ tiếp theo, mô hình không chỉ nhìn vào từ ngay trước nó, mà nó nhìn vào toàn bộ các từ đã xuất hiện và dùng cơ chế gọi là Self-Attention để xem từ nào quan trọng nhất. Ví dụ: Trong câu "Con mèo rượt đuổi con chuột vì nó chạy rất nhanh", cơ chế Attention sẽ giúp mô hình hiểu chữ "nó" ở đây có xác suất cao là đang chỉ "con chuột" chứ không phải "con mèo". Sau khi tính toán ngữ cảnh, mô hình sẽ quét qua toàn bộ kho từ vựng và gán cho mỗi từ một con số xác suất. Mô hình sẽ chọn từ tiếp theo dựa trên bảng xác suất này. Quá trình này lặp đi lặp lại liên tục cho đến khi nó tạo ra một ký tự đặc biệt ra lệnh dừng lại.



Hình 1.3: Cách hoạt động của LLM

Tuy nhiên LLM có một số nhược điểm:

- Dữ liệu huấn luyện rộng
- Độ chính xác không cao với miền cụ thể
- Có hiện tượng ảo giác (Hallucination)
- Dữ liệu chỉ cập nhật đến ngày train, dữ liệu cũ
- LLM không biết dữ liệu mới

- LLM không biết dữ liệu doanh nghiệp
- LLM không biết tài liệu người dùng
- Chi phí cho việc huấn luyện, tinh chỉnh lớn.

LLM CÓ MỘT SỐ NHƯỢC ĐIỂM



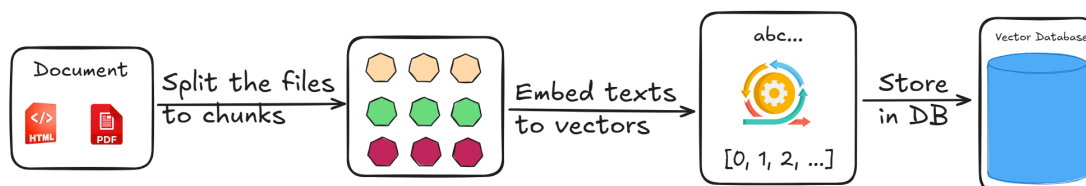
Hình 1.4: Một số nhược điểm của LLM

1.2.2 Tạo tăng cường truy xuất RAG (Retrieval Augmented Generation)

Mô hình ngôn ngữ lớn có thể đưa ra kết quả không chính xác trong thực tế, đối mặt với thách thức như hiểu được lập luận pháp lý phức tạp, đảm bảo tính minh bạch và tránh các phản hồi gây hiểu nhầm. Làm thế nào để mô hình ngôn ngữ lớn có thể trả lời các câu hỏi pháp luật một cách chính xác? Retrieval Augmented Generation (RAG) là một kỹ thuật được thiết kế để khắc phục hiện tượng ảo giác của LLM bằng cách cung cấp thêm nguồn tri thức bên ngoài trước khi sinh ra câu trả lời.

Ứng dụng trong đồ án

Trong phạm vi đồ án này, em sử dụng RAG để tăng cường khả năng truy xuất tri thức từ các văn bản pháp luật, giúp mô hình ngôn ngữ lớn đưa ra câu trả lời chính xác hơn, có căn cứ pháp lý rõ ràng và giảm thiểu hiện tượng ảo giác so với mô hình LLM truyền thống. Thay vì “đoán”, LLM sẽ dựa vào dữ liệu thật từ đó cung cấp nguồn dẫn chứng rõ ràng cho người dùng.



Extracting and Indexing

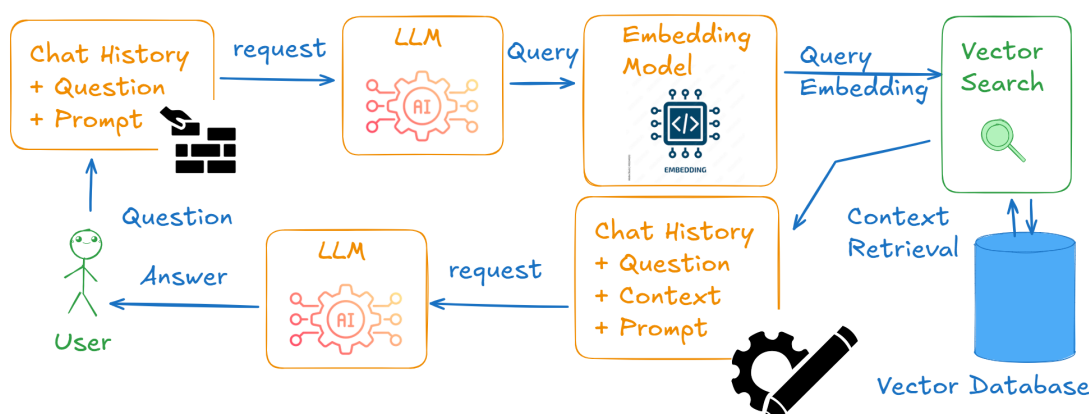
Hình 1.5: Sơ đồ RAG cơ bản (Extracting and Indexing)

Các bước cốt lõi để xây dựng một hệ thống RAG:

- Toàn bộ dữ liệu tri thức (tài liệu, văn bản luật, ...) được chia nhỏ thành các đoạn (chunks).
- Sử dụng các mô hình nhúng (embedding models) để chuyển đổi các đoạn văn bản này thành các biểu diễn vector.
- Lưu trữ các vector vào vector database để tối ưu hóa việc tìm kiếm không gian nhiều chiều.
- Hệ thống nhận yêu cầu từ người dùng dưới dạng ngôn ngữ tự nhiên.
- Câu truy vấn của người dùng cũng được mã hóa thành vector.
- Cơ chế truy xuất sẽ quét toàn bộ vector database để đo lường độ tương đồng ngữ nghĩa nhằm trích xuất ra các đoạn tài liệu liên quan nhất.
- Các đoạn văn bản liên quan vừa được truy xuất sẽ được ghép nối với câu truy vấn ban đầu của người dùng.
- Câu prompt đã được làm giàu ngữ cảnh sẽ được đưa vào LLM
- LLM sử dụng ngữ cảnh được cung cấp để sinh ra câu trả lời mạch lạc và có độ chính xác cao đối với nguồn dữ liệu.

Việc áp dụng kiến trúc RAG không chỉ giúp giảm đáng kể chi phí huấn luyện và bảo trì mô hình mà còn hạn chế hiện tượng tạo ra thông tin ảo giác thông qua việc dựa trên các tài liệu được truy xuất từ nguồn dữ liệu đáng tin cậy. Điều này giúp nâng cao độ chính xác, khả năng mở rộng và tính cập nhật của hệ thống.

Retrieval and Generation

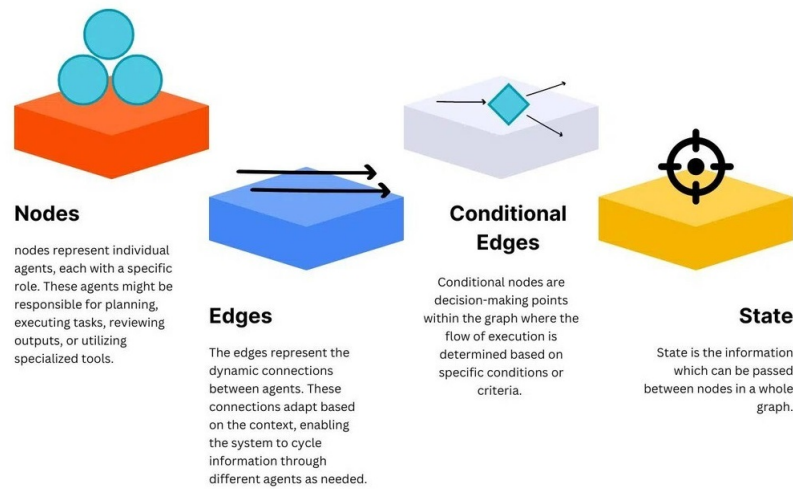


Hình 1.6: Sơ đồ RAG cơ bản (Retrieval and Generation)

Tuy nhiên, sử dụng RAG đơn giản thì độ chính xác chưa cao và vẫn có thể sai sót vì vậy, em sử dụng LangGraph xây dựng Agentic RAG để tăng độ chính xác và hạn chế ảo giác vì miền tư vấn pháp luật yêu cầu độ chính xác cao. Thay vì thực thi các bước theo quy trình tuyến tính như LangChain, LangGraph mô hình hóa luồng công việc dưới dạng một đồ thị gồm các thành phần chính sau:

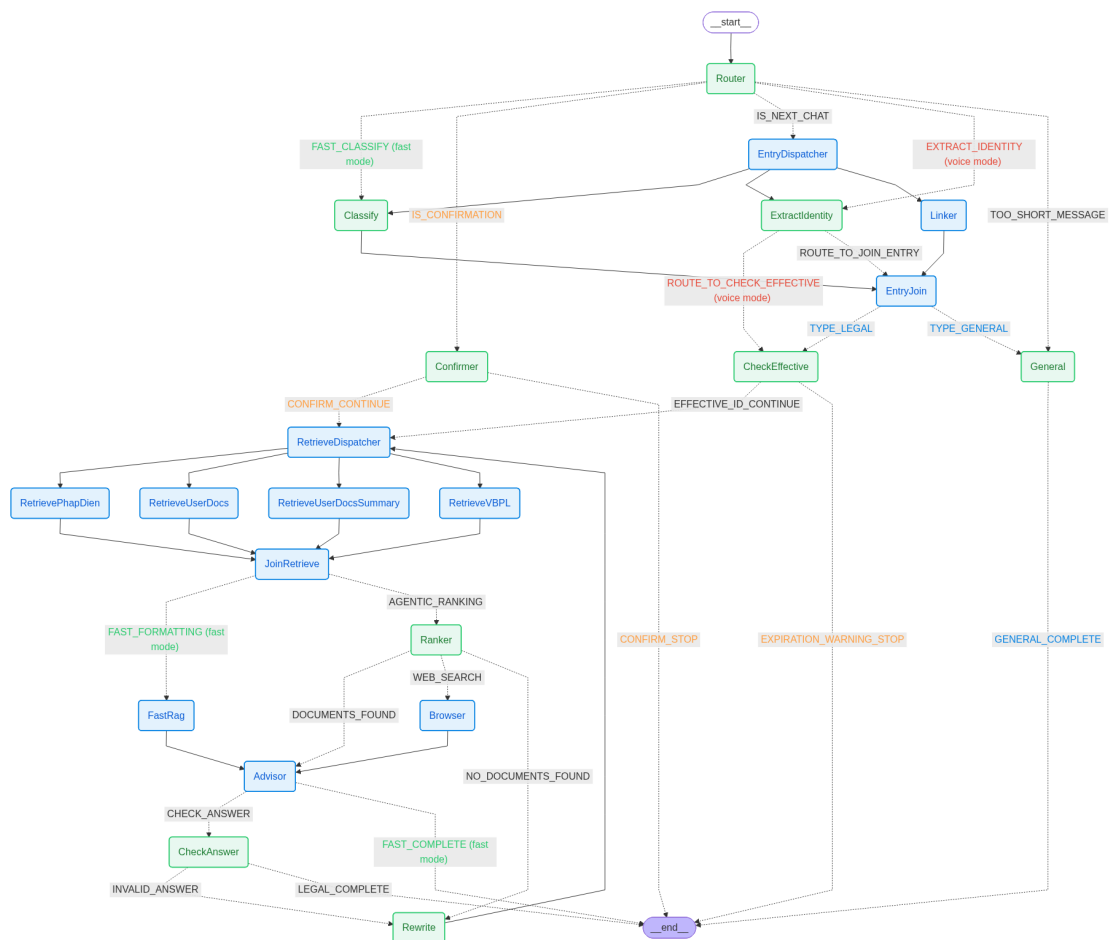
- Nodes (Nút): Đại diện cho các bước xử lý, tác vụ hoặc hành động cụ thể, chẳng hạn như gọi mô hình ngôn ngữ lớn (LLM), truy xuất cơ sở dữ liệu hoặc tương tác với người dùng.
- Edges (Cạnh): Xác định hướng chuyển tiếp giữa các nút. Các cạnh có thể là đường đi trực tiếp hoặc cạnh có điều kiện (conditional edges), cho phép lựa chọn nhánh thực thi dựa trên kết quả của nút trước đó.
- Cycles (Vòng lặp): Đây là đặc điểm nổi bật của LangGraph. Khác với kiến trúc đồ thị có hướng không chu trình, LangGraph cho phép tác nhân (agent) quay lại các nút đã thực hiện để đánh giá lại kết quả, tự sửa lỗi hoặc lặp lại quy trình nhiều lần cho đến khi đạt được mục tiêu mong muốn.

- **State (Trạng thái):** Đây là bộ nhớ dùng chung. Mọi nút (node) trong đồ thị đều có thể đọc dữ liệu từ State này và ghi kết quả mới của chúng vào đây.



Hình 1.7: Các khái niệm quan trọng của LangGraph

Sơ đồ Agentic RAG dự án này:



Hình 1.8: Sơ đồ LangGraph của dự án

Giải thích sơ đồ Agentic RAG:

- Hệ thống phân chia việc sử dụng thông minh giữa mô hình nhỏ cho luồng cơ bản và mô hình lớn cho tác vụ suy luận là chìa khóa để tối ưu cả chi phí, độ trễ và hiệu năng. (Nhìn vào sơ

đồ các Node màu xanh lá sử dụng mô hình nhỏ, các Node màu xanh dương sử dụng mô hình lớn)

- Mỗi tin nhắn đều đi qua **Router**. Node này kiểm tra điều kiện giúp định tuyến vào luồng chính xác theo từng trạng thái.
- Ở chế độ suy luận Agentic RAG đầy đủ thực hiện phân tích song song
 - **Linker** đọc URL nếu có.
 - **Classify** phân loại câu hỏi là pháp lý hay thông thường.
 - **Extract Identity** trích xuất số hiệu văn bản (VD: 100/2019/NĐ-CP).
- Kiểm tra hiệu lực pháp lý: Nếu câu hỏi thuộc nhóm **TYPE_LEGAL** và có số hiệu văn bản, hệ thống tra cứu trạng thái.
 - Văn bản đã hết hiệu lực hoàn toàn thì kết thúc ngay.
 - Văn bản hết hiệu lực một phần thì phát cảnh báo và chờ người dùng xác nhận ở lượt sau qua **Confirmer**.
- Truy xuất nguồn dữ liệu song song
 - Tài liệu người dùng (Milvus)
 - Tóm tắt tài liệu (Cloudflare R2)
 - Văn bản pháp luật (Pinecone)
 - Dữ liệu pháp điển (Qdrant)
- Tiếp tục tới **Ranker** để lọc tài liệu
 - Nếu có tài liệu hợp lý thì chuyển đến **Advisor** để tiến hành tạo câu trả lời.
 - Nếu không có tài liệu hợp lý thì chuyển đến **Rewrite** để tạo lại câu hỏi.
 - Sau khi thử tạo lại câu hỏi vẫn không có tài liệu hợp lý thì sẽ dùng **Browser** để lấy tài liệu tại nguồn thuvienphapluat.vn và luatvietnam.vn
- Vòng lặp tự kiểm tra chất lượng Sau khi **Advisor** trả lời, **Check Answer** đánh giá câu trả lời có hợp lý hay không?
 - Nếu không hợp lệ và chưa quá 2 lần thì xóa câu trả lời cũ, chuyển đến **Rewrite** để tạo lại câu hỏi và lặp lại toàn bộ luồng truy xuất.
 - Nếu câu trả lời hợp lệ hoặc đã quá số lần thử lại thì hệ thống kết thúc quá trình trả lời.
- Ở chế độ Fast RAG chỉ đi qua **Classify** để phân loại. Sau đó truy xuất dữ liệu lấy dữ liệu. Dùng **Advisor** trả lời và hệ thống kết thúc quá trình trả lời mà không cần bước kiểm tra.
- Ở chế độ giọng nói Voice, do đã có Agent nên không cần qua bước phân loại của LangGraph và người dùng không thể nhập URL nên hệ thống chuyển đến **Extract Identity** trích xuất số hiệu văn bản.

1.2.3 Kiến trúc vi dịch vụ MSA (Microservice Architecture)

Kiến trúc vi dịch vụ chia bài toán thành các thành phần nhỏ hơn được gọi là các dịch vụ.

- Mỗi dịch vụ tập trung vào một khả năng kinh doanh cụ thể.
- Các dịch vụ độc lập và giao tiếp với nhau thông qua hạ tầng mạng.

Một số lợi ích của Microservice:

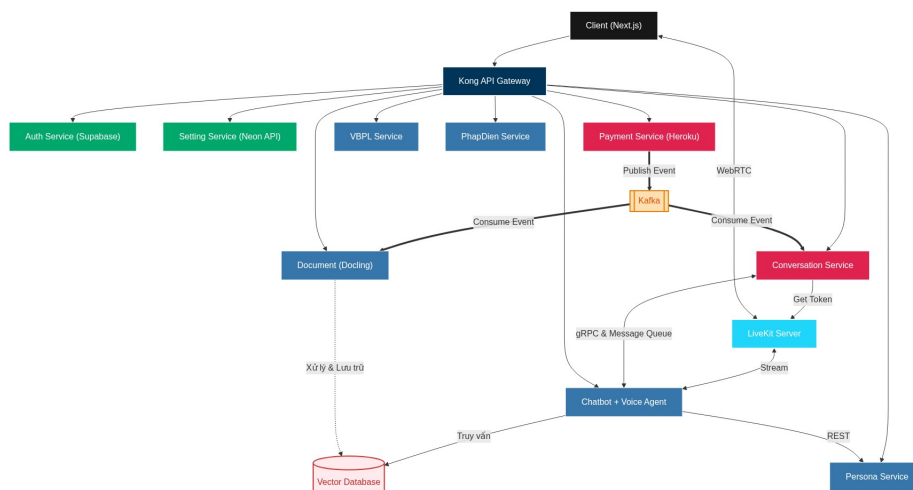
- Mỗi dịch vụ có các biến bí mật (secret) riêng.
- Không cần cài các thư viện, các phụ thuộc với dịch vụ không dùng
- Đa dạng ngôn ngữ lập trình khác nhau theo từng logic của từng dịch vụ

- Phân chia tài nguyên cho từng dịch vụ (Ví dụ Heroku giới hạn RAM, AI cần GPU)
- Giảm thời gian quá trình kiểm thử vì mỗi dịch vụ có nghiệp vụ khác nhau
- Vì các dịch vụ độc lập, dễ dàng triển khai
- Tăng khả năng mở rộng hệ thống



Hình 1.9: Giới thiệu Microservice

Kiến trúc tổng quan các dịch vụ của dự án:



Hình 1.10: Kiến trúc tổng quan các dịch vụ của dự án

1.2.4 Thiết kế hướng miền DDD (Domain Driven Design)

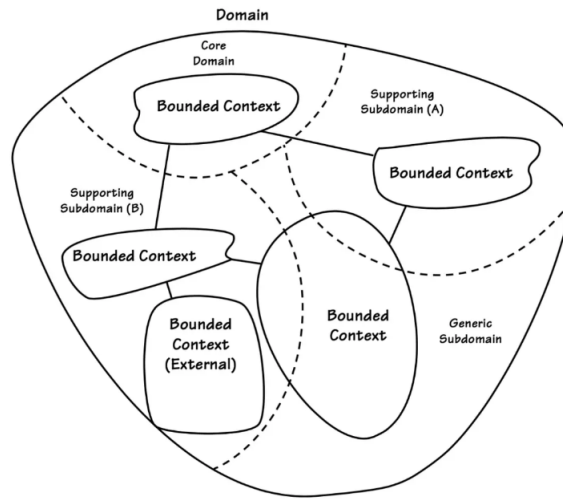
Thiết kế hướng miền là một phương pháp tiếp cận thiết kế phần mềm do Eric Evans giới thiệu, tập trung vào việc thấu hiểu và mô hình hóa sâu sắc lĩnh vực kinh doanh của tổ chức. Thay vì để logic nghiệp vụ bị lẫn lộn với các yếu tố kỹ thuật (như cơ sở dữ liệu, hạ tầng mạng), DDD đặt nghiệp vụ làm trung tâm để đưa ra giải pháp linh hoạt, dễ bảo trì và dễ mở rộng.

Cốt lõi của DDD được chia thành hai nhóm mẫu chính: **Các mẫu chiến lược (Strategic Patterns)** và **Các mẫu kỹ thuật (Tactical Patterns)**.

Các mẫu chiến lược (Strategic Patterns)

Nhóm mẫu này giúp phân chia một hệ thống lớn và phức tạp thành các phần nhỏ hơn với ranh giới rõ ràng, tập trung vào kiến trúc tổng thể.

- **Miền (Domain) & Miền phụ (Sub-Domain):** Phạm vi kiến thức hoặc vấn đề mà phần mềm giải quyết. Một miền lớn được chia thành 3 loại miền phụ:
 - Miền phụ cốt lõi (Core Subdomain)
 - Miền phụ hỗ trợ (Supporting Subdomain)
 - Miền phụ chung (Generic Subdomain)



Hình 1.11: Các loại miền phụ trong Domain Driven Design

- **Chuyên gia miền (Domain Expert):** Người hiểu sâu sắc nhất về quy trình kinh doanh thực tế, làm việc chặt chẽ với đội ngũ phát triển.
- **Ngôn ngữ chung (Ubiquitous Language):** Ngôn ngữ thống nhất được sử dụng xuyên suốt bởi cả đội ngũ kỹ thuật và chuyên gia miền, từ giao tiếp hàng ngày đến việc đặt tên trong mã nguồn.
- **Bối cảnh bị giới hạn (Bounded Context):** Xác định ranh giới rõ ràng cho một phần của hệ thống, giúp các nhóm hoạt động độc lập và giữ cho mô hình nhất quán trong phạm vi đó.
- **Bản đồ bối cảnh (Context Maps) & Các mối quan hệ:** Thể hiện trực quan cách các Bounded Context giao tiếp với nhau.

Các mẫu kỹ thuật (Tactical Patterns)

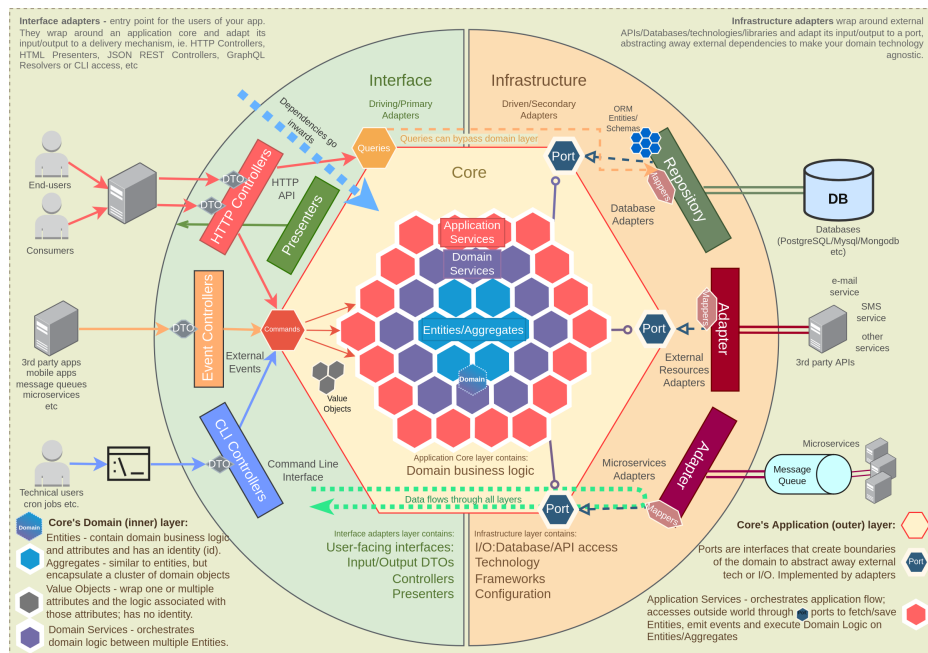
Nhóm mẫu này đóng vai trò hiện thực hóa các khái niệm nghiệp vụ vào thiết kế mã nguồn hệ thống.

- **Đối tượng thực thể (Entities Objects):** Các đối tượng có **định danh duy nhất (ID)** tồn tại xuyên suốt vòng đời. Thực thể quản lý trạng thái và có thể bị thay đổi theo thời gian.
- **Đối tượng giá trị (Value Objects):** Các đối tượng không có định danh duy nhất, dùng để đại diện cho một đặc điểm (ví dụ: Địa chỉ, Giá tiền). Chúng được so sánh qua các thuộc tính và thường được thiết kế ở dạng bất biến (immutable).
- **Tổng hợp (Aggregates):** Một nhóm các Thực thể và Đối tượng giá trị được gom lại thành một khối thống nhất để đảm bảo tính toàn vẹn dữ liệu. Giao tiếp với bên ngoài bắt buộc phải thông qua một thực thể gốc (Aggregate Root).
- **Nhà máy (Factory):** Đóng gói quy trình khởi tạo phức tạp của các đối tượng hoặc Aggregates để đảm bảo chúng luôn được tạo ra ở trạng thái hợp lệ.
- **Kho lưu trữ (Repository):** Trừu tượng hóa lớp tương tác với cơ sở dữ liệu. Nó hoạt động như một bộ sưu tập lưu trữ các đối tượng Aggregates trong bộ nhớ, giúp mô hình miền tách biệt hoàn toàn khỏi công nghệ lưu trữ.
- **Dịch vụ miền (Domain Service):** Nơi chứa các quy tắc hoặc hành vi nghiệp vụ phức tạp liên quan đến nhiều đối tượng, không phù hợp để đặt vào bên trong một Thực thể hay Đối tượng giá trị cụ thể.

Kiến trúc hệ thống trong DDD

Để bảo vệ lớp miền cốt lõi khỏi các công nghệ bên ngoài, báo cáo áp dụng các kiến trúc phân lớp:

- **Kiến trúc phân lớp (Layered Architecture):** Thường chia làm 4 lớp cơ bản: Giao diện người dùng (User Interface) → Ứng dụng (Application) → Lớp Miền (Domain - trung tâm) → Hạ tầng (Infrastructure).
- **Kiến trúc lục giác (Hexagonal Architecture / Ports and Adapters):** Đây là kiến trúc được áp dụng chính. Đặt lớp Miền (Domain) ở vị trí trung tâm, cách ly hoàn toàn với các yếu tố bên ngoài. Mọi giao tiếp với cơ sở dữ liệu, API, hoặc giao diện đều thông qua các **Cổng (Ports)** và **Bộ điều hợp (Adapters)**. Kiến trúc này giúp hệ thống cực kỳ dễ kiểm thử, linh hoạt thay đổi công nghệ (Database, Framework) mà không làm ảnh hưởng đến logic nghiệp vụ cốt lõi.
- **Nguyên tắc đảo ngược phụ thuộc (Dependency Inversion):** Đảo ngược phụ thuộc là nguyên tắc cốt lõi giúp bảo vệ và cô lập lớp Miền (Domain Layer). Cơ chế hoạt động: Lớp Miền sẽ định nghĩa các giao diện (chính là các Cổng - Ports trong Kiến trúc lục giác) để yêu cầu những gì nó cần. Các module cấp thấp (như CSDL, hàng đợi tin nhắn) sẽ triển khai các giao diện này (đóng vai trò là Bộ điều hợp - Adapters) và cắm vào lớp Miền như những “plugin” độc lập.



Hình 1.12: Tổng quan kiến trúc lục giác

TÀI LIỆU THAM KHẢO

- [1] <https://vi.wikipedia.org/wiki/FPT>
- [2] Tài liệu đào tạo hội nhập của Công ty TNHH FPT IS